

Laboratorio 3: Clasificación de texto con Naive Bayes

1. Conjuntos de entrenamiento, validación y prueba

El corpus de 1,138 documentos se dividió en tres conjuntos con estratificación por categoría. El conjunto de entrenamiento quedó con 796 documentos (70%), el de validación con 171 (15%) y el de prueba con 171 (15%). La distribución por categoría quedó prácticamente igual en los tres conjuntos: Alianzas 21.6%, Innovacion 13.3%, Macroeconomia 28.0%, Otra 11.3%, Regulaciones 12.4%, Reputacion 2.3% y Sostenibilidad 11.1%.

La estratificación es importante porque el corpus está desbalanceado. Macroeconomia tiene 340 documentos y Reputacion solo 26. Sin estratificar, el conjunto de prueba podría haber quedado con muy pocos ejemplos de las categorías más pequeñas, haciendo la evaluación poco representativa.

El conjunto de entrenamiento es del que el modelo aprende todo y donde calcula las probabilidades. El de validación se usa para comparar versiones del modelo mientras se desarrolla, por ejemplo para ver si BoW funciona mejor que TF-IDF o qué tamaño de vocabulario conviene. El de prueba se guarda para el final y se usa una sola vez, para medir el desempeño real con datos que el modelo nunca vio. Si uno toma decisiones basándose en los resultados del conjunto de prueba, la medición deja de ser honesta porque el modelo estaría aprendiendo de esos datos también, aunque sea indirectamente.

2. Construcción del clasificador Naive Bayes

Se entrenaron dos modelos MultinomialNB, uno usando Bag of Words y otro usando TF-IDF. Ambos vectorizadores se ajustaron únicamente con el conjunto de entrenamiento (vocabulario de 19,157 palabras) y luego se aplicaron a validación y prueba con ese mismo vocabulario para evitar fuga de información.

$P(c)$ es qué tan común es cada categoría en el corpus de entrenamiento. Por ejemplo, como Macroeconomia representa el 28% de los documentos, su $P(c) \approx 0.28$, lo que hace que el modelo ya tenga cierta inclinación hacia esa categoría antes de ver el texto. $P(w_i|c)$ es la probabilidad de que aparezca una palabra específica dado que el documento pertenece a cierta categoría, es decir, qué tan seguido aparece esa palabra dentro de los documentos de esa clase. Para clasificar un documento nuevo, el modelo calcula un puntaje por cada categoría combinando $P(c)$ con $P(w_i|c)$ de cada palabra del documento, asumiendo que las palabras son independientes entre sí. La categoría con el puntaje más alto gana.

3. Entrenamiento y evaluación

A continuación se presentan los reportes de clasificación del modelo BoW en validación y prueba:

BoW – Validación				
	precision	recall	f1-score	support
Alianzas	0.83	0.78	0.81	37
Innovacion	0.68	1.00	0.81	23
Macroeconomia	0.82	0.85	0.84	48
Otra	0.75	0.63	0.69	19
Regulaciones	0.82	0.67	0.74	21
Reputacion	1.00	0.25	0.40	4
Sostenibilidad	0.89	0.84	0.86	19
accuracy			0.80	171
macro avg	0.83	0.72	0.73	171
weighted avg	0.81	0.80	0.79	171

BoW – Prueba				
	precision	recall	f1-score	support
Alianzas	0.83	0.78	0.81	37
Innovacion	0.74	1.00	0.85	23
Macroeconomia	0.88	0.92	0.90	48
Otra	0.90	0.45	0.60	20
Regulaciones	0.94	0.76	0.84	21
Reputacion	1.00	0.25	0.40	4
Sostenibilidad	0.67	1.00	0.80	18
accuracy			0.82	171
macro avg	0.85	0.74	0.74	171
weighted avg	0.84	0.82	0.81	171

El modelo TF-IDF tuvo un rendimiento menor, ya que el accuracy de 0.52 en validación y 0.56 en prueba. Categorías como Otra, Regulaciones y Reputacion tuvieron precision y recall en 0, lo que significa que el modelo casi nunca las predijo. Esto ocurre probablemente porque los valores TF-IDF son decimales normalizados y no se combinan bien con el suavizado de Laplace de MultinomialNB, que espera conteos de palabras.

Comparando entrenamiento vs validación/prueba: BoW tuvo accuracy de 0.9523 en entrenamiento y bajó a 0.7953 en validación y 0.8187 en prueba. Esa caída es señal de cierto overfitting. Con 19,157 palabras de vocabulario y solo 796 documentos de entrenamiento, el modelo aprende patrones muy específicos que no necesariamente se repiten en datos nuevos. El mejor modelo fue BoW por bastante margen. TF-IDF penaliza palabras que se repiten mucho en el corpus, pero en noticias de economía y negocios palabras como "empresa", "banco" o "mercado" son las que mejor distinguen entre categorías, y TF-IDF termina borrando esas señales.

4. Implementación propia de Naive Bayes

Se programó Naive Bayes desde cero calculando $P(c)$ como la proporción de documentos por clase, $P(w_i|c)$ con suavizado de Laplace ($\alpha=1.0$) para evitar probabilidades de cero, y usando logaritmos para evitar underflow al multiplicar muchas probabilidades pequeñas.

Los resultados en el conjunto de prueba fueron exactamente iguales a los de MultinomialNB de sklearn: accuracy 0.8187, F1 macro 0.7425 y F1 ponderado 0.8081 en ambos casos, coincidiendo hasta el último decimal. Esto confirma que la implementación propia calcula exactamente lo mismo que sklearn internamente.

5. Análisis de errores

El par más confundido fue Otra y Sostenibilidad: 4 documentos reales de Otra fueron clasificados como Sostenibilidad. Revisando esos 4 documentos, todos hablan de BBVA en

contextos de sostenibilidad bancaria, financiamiento sostenible, rankings o foros. Comparten mucho vocabulario con Sostenibilidad ("sostenible", "sostenibilidad", "banco") aunque técnicamente estén etiquetados como Otra porque el foco de la noticia es otro. Esto coincide con lo observado en el Laboratorio 2, donde el documento índice 0 de categoría Otra salía como muy similar por coseno a documentos de Sostenibilidad.

Las palabras con mayor $P(w|c)$ tienen sentido en su mayoría. Para Macroeconomía salen inflación (0.01068), precio (0.00799), tasa (0.00370) y mes (0.00399). Para Alianzas la palabra más probable es directamente "alianza" (0.00539). Para Innovación, en cambio, salen bbva (0.01198), banco (0.00482) y poder (0.00593), que son palabras genéricas o nombres de empresa que no son realmente representativas del tema innovación.

6. Efecto de la representación de texto

Al limitar el vocabulario a 1,000 palabras el modelo obtuvo accuracy 0.8655 y F1 macro 0.8565. Con 5,000 palabras, el accuracy quedó igual pero el F1 macro subió a 0.8691. Ambos superan al vocabulario completo de 19,157 palabras (accuracy 0.8187, F1 macro 0.7425). Con solo 796 documentos de entrenamiento y casi 20,000 palabras, muchas aparecen muy pocas veces y sus probabilidades son poco confiables. Al quedarse solo con las más frecuentes, las estimaciones son más estables.

Con bigramas el vocabulario creció de 19,157 a 185,042 términos y el desempeño empeoró a accuracy 0.7719 y F1 macro 0.6773 en solo 0.29 segundos de entrenamiento.

Tabla comparativa de todos los modelos:

Tabla comparativa de todos los modelos:			
Representación	Vocabulario	F1 macro	Accuracy
BoW - max_features=5000	5000	0.8691	0.8655
BoW - max_features=1000	1000	0.8565	0.8655
BoW - vocabulario completo	19157	0.7425	0.8187
Implementación propia (BoW)	19157	0.7425	0.8187
BoW - unigramas + bigramas	185042	0.6773	0.7719
TF-IDF - vocabulario completo	19157	0.3362	0.5556

7. Análisis

1. ¿Por qué Naive Bayes es probabilístico y no basado en distancias?

Naive Bayes calcula directamente la probabilidad de que un documento pertenezca a cada categoría, aprendiendo esas probabilidades de datos etiquetados. La similitud coseno del Laboratorio 2 solo compara la dirección de dos vectores sin aprender nada. La ventaja de Naive Bayes es que clasifica directamente sin necesitar un documento de referencia para comparar. Una limitación visible en este laboratorio es que el modelo depende mucho de que las probabilidades estén bien estimadas: con TF-IDF los resultados fueron muy malos (accuracy 0.56) porque los valores decimales normalizados no encajan bien con el suavizado de Laplace.

2. ¿Por qué funcionó bien a pesar del supuesto de independencia?

Aunque el modelo trata cada palabra como independiente, en la práctica funciona porque las señales se suman. Cuando hay muchas palabras características de una categoría, como inflación, precio, tasa y mes para Macroeconomía, cada una aporta un poco al puntaje. Aunque ninguna palabra sola sea perfecta, la suma termina inclinando la balanza hacia la categoría

correcta. Además, con datos dispersos como los de este corpus, Naive Bayes es robusto porque no necesita estimar correlaciones entre palabras, que con pocos datos serían muy poco confiables.

3. ¿Qué parte de la implementación propia fue más difícil?

La parte más difícil fue el suavizado de Laplace combinado con algunos de los logaritmos. Ya que, entender por qué hay que sumar α tanto al numerador como al denominador y todos esos procedimientos algebraicos (y multiplicar por el tamaño del vocabulario en el denominador) no es inmediato, y cualquier error ahí hace que las probabilidades no sean correctas. Además, personalmente me gusta bastante entender bien qué es lo que está sucediendo y cómo lo hace. Por lo que, se me dificultó bastante entender todo el proceso. Ver que los resultados salieran idénticos hasta el sexto decimal fue la confirmación de que el cálculo estaba bien hecho pero para llegar a entender todo, me llevó bastantes horas de razonamiento y análisis.

4. ¿Qué categorías fueron más fáciles y cuáles más difíciles?

Las categorías más fáciles fueron Macroeconomía (F1 0.90 en prueba), Innovación (F1 0.85) y Sostenibilidad (F1 0.80). Las más difíciles fueron Reputación (F1 0.40, solo 4 documentos en el conjunto de prueba) y Otra (F1 0.60). Lo mismo pasó en el laboratorio 2, ya que la similitud coseno el documento índice 0 de Otra salía mezclado con documentos de Sostenibilidad, entonces lo que sugiere que esa confusión es estructural del corpus y no solo un problema del clasificador.

5. ¿En qué momento se usó validación y en qué momento prueba?

El conjunto de validación se usó mientras se desarrollaba el modelo, específicamente para comparar BoW versus TF-IDF y decidir cuál representación funcionaba mejor. El conjunto de prueba se usó una sola vez al final, para la evaluación definitiva y el análisis de errores. Es importante no mezclarlos porque si se ajustan decisiones mirando el conjunto de prueba, este deja de ser una medición honesta del desempeño real con datos nuevos.

6. ¿Qué riesgos habría si este clasificador se pusiera en producción?

El primer riesgo es el desbalance de clases: Reputación tiene solo 26 documentos en todo el corpus y el modelo apenas la detecta (recall 0.25), por lo que en producción esas noticias se clasificarían casi siempre mal. El segundo riesgo es el vocabulario fijo: el modelo no reconoce términos nuevos que aparezcan en noticias futuras, lo que puede ir empeorando con el tiempo. El tercer riesgo es que si aparece una categoría nueva el modelo no podría predecirla nunca. Como mitigaciones: reentrenar periódicamente con datos nuevos, monitorear el desempeño en producción y marcar como "baja confianza" las predicciones donde ninguna categoría tenga un puntaje claramente dominante para revisión humana.

LINK AL GITHUB: <https://github.com/luispegr25/NLP>